

## Product Requirements Document

Version 1.0 · April 2026 · Internal — Customer Success

|         |                              |          |                        |      |       |
|---------|------------------------------|----------|------------------------|------|-------|
| AUTHOR  | Product Analyst Candidate    | STATUS   | Final — Submission     |      |       |
| PRODUCT | ClearQuote CS Dashboard v1.0 | AUDIENCE | ClearQuote Engineering | CS   | Team, |
| DATE    | 28 April 2026                | STACK    | React 18 · TypeScript  | Vite |       |
|         |                              |          | Recharts · D3          |      |       |

### 1. Executive Summary

The ClearQuote Customer Success team currently tracks customer health, product usage, and support activity across disconnected spreadsheets and manual reports. This creates visibility gaps, delayed escalations, and wasted CSM time on data wrangling instead of customer engagement.

This document defines requirements for a browser-based internal dashboard that centralises all CS intelligence into a single, always-current view. The dashboard enables CSMs to identify at-risk accounts in seconds, surface usage anomalies before they become churn events, and manage support workload with full context — without leaving a single screen.

### 2. Problem Statement

#### CURRENT STATE

CSMs maintain 3–5 separate spreadsheets covering customer health, inspection volumes, open tickets, and fleet data. These are updated manually, typically weekly, and are never fully in sync with one another.

|                      |                                                                                                                                                                                                                                                |
|----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>CORE PAIN</b>     | There is no single source of truth. A CSM preparing for a renewal call must open multiple tabs, cross-reference data by hand, and rely on memory for context they should have at a glance.                                                     |
| <b>BUSINESS RISK</b> | Usage drops and open high-priority tickets — two leading churn indicators — are regularly missed until the customer raises them. The average time-to-response for a high-priority ticket is currently 2.4 days; industry benchmark is 4 hours. |
| <b>OPPORTUNITY</b>   | A unified dashboard consolidating customers, usage, support, and fleet data into one view is estimated to save each CSM 3–4 hours per week and reduce missed escalations by >60%.                                                              |

### 3. User Personas

|                    |                                                                                                                                                                                                                                                                                         |
|--------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>CSM</b>         | Customer Success Manager · Primary User                                                                                                                                                                                                                                                 |
| <b>GOALS</b>       | <ul style="list-style-type: none"> <li>&gt;• Monitor health of assigned book of business (8–12 accounts)</li> <li>• Prepare for renewal and QBR meetings with minimal prep time</li> <li>• Detect usage drops or ticket spikes before customers escalate</li> </ul>                     |
| <b>PAIN POINTS</b> | <ul style="list-style-type: none"> <li>&gt;• Spends 45 min/day manually compiling account status</li> <li>• No alert when a customer's usage drops or a ticket goes stale</li> <li>• Has to switch between 4 tools to get a complete account picture</li> </ul>                         |
| <b>CS Lead</b>     | Head of Customer Success · Secondary User                                                                                                                                                                                                                                               |
| <b>GOALS</b>       | <ul style="list-style-type: none"> <li>&gt;• Track overall portfolio health and MRR at risk</li> <li>• Identify which CSMs need support or escalation assistance</li> <li>• Report weekly CS metrics to the executive team</li> </ul>                                                   |
| <b>PAIN POINTS</b> | <ul style="list-style-type: none"> <li>&gt;• No roll-up view of at-risk accounts across the full portfolio</li> <li>• Relies on CSMs to self-report, introducing lag and bias</li> <li>• Cannot easily identify systemic issues (e.g. a telematics provider causing tickets)</li> </ul> |
| <b>Product</b>     | Product Analyst / Product Manager · Tertiary User                                                                                                                                                                                                                                       |
| <b>GOALS</b>       | <ul style="list-style-type: none"> <li>&gt;• Understand which customers are adopting which product features</li> <li>• Correlate damage rates and inspection volumes with fleet composition</li> <li>• Identify segments for targeted product experiments</li> </ul>                    |
| <b>PAIN POINTS</b> | <ul style="list-style-type: none"> <li>&gt;• Usage data is aggregated monthly and not sliceable by customer attributes</li> <li>• No easy way to join fleet metadata with product usage metrics</li> </ul>                                                                              |

## 4. Requirements Prioritisation (MoSCoW)

Features are prioritised using the MoSCoW framework. All **Must Have** items are implemented in v1. **Should Have** items are implemented where feasible within the time constraint. **Could Have** and **Won't Have** items are explicitly deferred and documented as trade-offs.

| Priority    | Feature                                              | Rationale                                                             |
|-------------|------------------------------------------------------|-----------------------------------------------------------------------|
| Must Have   | 4-tab layout (Overview, Usage, Support, Fleet)       | Core deliverable — directly maps to CS team's daily workflow          |
| Must Have   | Customer health status (Healthy / Warning / At Risk) | Primary signal for CS team to triage their book of business           |
| Must Have   | US map with customer pins                            | Spatial context instantly surfaces geographic clustering of risk      |
| Must Have   | Ticket list with priority, age & CSAT                | Support ageing is the highest urgency item for CS managers            |
| Must Have   | Search & filter on all tabs                          | 30 customers is manageable; scalability required filter from day 1    |
| Should Have | Monthly inspection & damage rate charts              | Leading indicator of churn — usage drop precedes cancellation         |
| Should Have | Fleet composition & EV adoption breakdown            | CSMs need this for renewal conversations and upsell positioning       |
| Should Have | API usage trend per customer                         | Identifies over/under-utilisation before renewal discussions          |
| Could Have  | Google SSO (@clearquote.io only)                     | Improves security; deprioritised as internal tool, low threat surface |
| Could Have  | Customer detail drill-down modal                     | Useful but single-page view sufficient for initial version            |
| Won't Have  | Real-time data sync / webhooks                       | Dummy data scope; architecture decision for production phase          |
| Won't Have  | Email / Slack alert automation                       | Out of scope for v1; can be added once health scoring is validated    |

## 5. Solution Overview

### 5.1 Four-Tab Layout

| TAB             | PURPOSE                                    | KEY COMPONENTS                                                                   |
|-----------------|--------------------------------------------|----------------------------------------------------------------------------------|
| <b>Overview</b> | Portfolio-level health at a glance         | KPI strip · US map pins · Searchable / filterable customer table                 |
| <b>Usage</b>    | Product adoption and inspection trends     | 15-month line charts · Active drivers bar chart · API trend · Per-customer table |
| <b>Support</b>  | Ticket workload and CSAT management        | Ticket table with ageing · Priority / channel / status filters · Star ratings    |
| <b>Fleet</b>    | Fleet composition and tooling intelligence | Vehicle type donut · Telematics / FMS breakdowns · Age distribution · EV % bars  |

## 5.2 Health Scoring Logic

Customer health is computed deterministically from three signals, producing one of three states: **Healthy** (score > 70), **Warning** (score 41–70), or **At Risk** (score <= 40).

|                  |                                            |
|------------------|--------------------------------------------|
| BASE SCORE       | 100 points                                 |
| USAGE TREND DOWN | –30 points (primary churn signal)          |
| OPEN TICKETS > 3 | –25 points (support overload indicator)    |
| CSAT < 3.0       | –20 points (satisfaction threshold breach) |

## 5.3 Data Model

All data is stored in five typed JSON files in **src/data/** and joined in the browser at render time using customer ID as the foreign key. No backend is required for v1.

| FILE           | RECORDS | KEY FIELDS                                           | JOINS TO          |
|----------------|---------|------------------------------------------------------|-------------------|
| customers.json | 30      | id, name, tier, mrr, csm, lat/lng, health            | All tables via id |
| usage.json     | 30      | cid, inspections, drivers, damage, api, trend        | customers.id      |
| monthly.json   | 15      | month, inspections, damageRate, apiCalls             | — (aggregate)     |
| tickets.json   | 20      | id, cid, issue, priority, channel, status, age, csat | customers.id      |
| fleet.json     | 30      | cid, vehicles, vtype, age, telem, fms, ev            | customers.id      |

## 6. Design Decisions & Trade-offs

### Inline styles over Tailwind CSS

Tailwind requires a PostCSS build step. In a Vite + React setup, misconfigurations silently produce unstyled HTML. Inline styles with a shared design token file (styles.ts) guarantee correct rendering with zero build dependency, and are equally maintainable at this codebase scale.

### Client-side data joining over a backend API

30 customers × 5 tables is ~5 KB of JSON — negligible for the browser. Introducing a backend adds deployment complexity (hosting, auth, CORS) with no performance benefit at this scale. The architecture is easily migrated to an API layer when real data sources are connected.

### Static JSON over a live database

The brief explicitly provides dummy data. Wiring to a real database or CRM (e.g. Salesforce, HubSpot) is the natural next step but is out of scope for v1. The data shape is designed to be a drop-in replacement for real API responses.

### Google SSO deferred

SSO is marked optional in the brief. Implementing OAuth2 with domain restriction adds ~4 hours of work and introduces a hard dependency on Google Cloud Console credentials. For a local demo / assessment context this is a poor time-to-value trade. The architecture supports adding it via a middleware auth provider (e.g. Auth0, NextAuth) with minimal refactoring.

### Health score computed client-side

Health scoring is a pure function of three fields available in the JSON data. Running it in the browser avoids a backend round-trip and makes the logic fully transparent and testable. In production this would move server-side to prevent clients from reverse-engineering scoring weights.

### Recharts over Chart.js or D3 directly

Recharts provides a React-idiomatic declarative API for the chart types needed (line, bar, pie). D3 is used directly only for the US map where its geo-projection capabilities have no equal. Mixing both libraries is intentional: use the right tool for each job.

## 7. What's Next (v2 Roadmap)

| FEATURE                                                                                                                            | PRIORITY |
|------------------------------------------------------------------------------------------------------------------------------------|----------|
| <b>Google SSO</b> — Restrict access to @clearquote.io domain via OAuth2 / Auth0.                                                   | High     |
| <b>Real data integration</b> — Replace JSON with API calls to CRM (Salesforce/HubSpot) and product analytics (Amplitude/Mixpanel). | High     |

| FEATURE                                                                                                                          | PRIORITY |
|----------------------------------------------------------------------------------------------------------------------------------|----------|
| <b>Customer drill-down</b> — Click any row to open a full account profile modal with all signals in one view.                    | Medium   |
| <b>Alert notifications</b> — Slack / email webhook when a customer crosses a health score threshold or a ticket exceeds SLA age. | Medium   |
| <b>CSM assignment management</b> — Reassign customers between CSMs directly from the dashboard.                                  | Medium   |
| <b>Export to CSV / PDF</b> — One-click account health report for exec updates and QBR preparation.                               | Low      |
| <b>Dark mode</b> — Design token system makes this a one-day implementation.                                                      | Low      |

## 8. Success Metrics

|                  |                                                                                                                             |
|------------------|-----------------------------------------------------------------------------------------------------------------------------|
| ADOPTION         | 100% of CSMs using the dashboard as their primary account view within 4 weeks of launch                                     |
| TIME SAVED       | Reduction in weekly account prep time from ~45 min to <15 min per CSM (measured via time-tracking survey)                   |
| ESCALATION SPEED | Mean time-to-identify for at-risk accounts reduced from >7 days to same-day                                                 |
| TICKET SLA       | High-priority ticket first-response time reduced from 2.4 days to <4 hours                                                  |
| DATA ACCURACY    | Zero discrepancies between dashboard health scores and CSM manual assessment in monthly audit (measured for first 3 months) |